

Relatório Técnico: Aplicação de Apoio à Proteção de Animais da UEFS

Gabriel Oliveira de Freitas

Departamento de Exatas (DEXA) – Universidade Estadual de Feira de Santana (UEFS), Av. Transnordestina, s/n - Novo Horizonte - CEP 44036-900 Feira de Santana - Bahia - Brasil

goliveira0403@gmail.com

Abstract. This report details the development of an application to support animal protection on the Universidade Estadual de Feira de Santana (UEFS) campus. The project addresses the need to replace manual animal data management with an efficient, automated system. A Java console application was developed, structured using the Model-View-Controller (MVC) architectural pattern. The system allows for the complete management of three central entities: Animals, Tutor Persons, and Responsible Sectors, supporting all CRUD (Create, Read, Update, and Delete) operations. Technical features include data validation, comprehensive unit testing with JUnit 5, and data persistence in JSON files using the Jackson library. The project was successfully completed, resulting in a functional, tested, and documented system that meets all proposed technical requirements.

Resumo. Este relatório detalha o desenvolvimento de uma aplicação de apoio à proteção dos animais no campus da Universidade Estadual de Feira de Santana (UEFS). O projeto atende à necessidade de substituir o controle manual de informações dos animais por um sistema automatizado e eficiente. Foi desenvolvida uma aplicação de console em Java, estruturada no padrão arquitetural Model-View-Controller (MVC). O sistema permite o gerenciamento completo de três entidades centrais: Animais, Pessoas Tutoras e Setores Responsáveis, suportando todas as operações de CRUD (Create, Read, Update, Delete). Dentre as funcionalidades técnicas, destacam-se a validação de dados, o desenvolvimento completo de testes de unidade com JUnit 5 e a persistência de dados em arquivos JSON com a biblioteca Jackson. O projeto foi concluído com sucesso, resultando em um sistema funcional, testado e documentado que cumpre todos os requisitos técnicos propostos.

1. Introdução

No campus da Universidade Estadual de Feira de Santana (UEFS), a convivência entre a comunidade acadêmica e diversos animais, como cães e gatos, é uma realidade constante. Para garantir o bem-estar desses animais, a universidade designou setores específicos para se responsabilizarem por seus cuidados, com o apoio de pessoas tutoras voluntárias.

Atualmente, o controle dessas informações é realizado de forma manual, o que gera dificuldades na organização de dados essenciais para cuidados veterinários,

alimentação, processos de adoção e monitoramento geral. A falta de um sistema centralizado torna tarefas simples, como consultar o histórico de um animal ou a lista de tutores de um setor, em processos lentos e suscetíveis a erros.

Diante deste contexto, este projeto propõe e implementa a "Aplicação de Apoio à Proteção de Animais da UEFS". Trata-se de um sistema de gerenciamento via console, desenvolvido em Java, projetado para automatizar e centralizar as operações de cadastro, consulta, atualização e exclusão (CRUD) das entidades Animal, PessoaTutora e SetorResponsavel. A utilização da Programação Orientada a Objetos e do padrão MVC permitiu a criação de um modelo de dados coeso e de um produto completo, de fácil manutenção e que atende diretamente às necessidades de gestão da universidade.

2. Fundamentação Teórica

A construção desta aplicação foi guiada por conceitos consolidados de algoritmos e programação, garantindo um código organizado, de fácil manutenção e confiável. Os pilares teóricos do projeto foram a Programação Orientada a Objetos (POO), o padrão de arquitetura Model-View-Controller (MVC) e a Persistência de Dados.

2.1. Programação Orientada a Objetos (POO)

A POO foi fundamental para modelar as entidades do sistema e suas interações. Os principais conceitos aplicados foram:

- **Encapsulamento:** Todos os atributos das classes do model (Animal, PessoaTutora, etc.) foram declarados como private. O acesso e a modificação desses dados são controlados por métodos públicos get e set, protegendo a integridade do estado interno dos objetos e garantindo que as regras de negócio sejam aplicadas de forma consistente.
- **Composição:** Em vez de herança, o projeto utilizou a composição para construir objetos complexos. As classes PessoaTutora e SetorResponsavel, por exemplo, "têm um" Endereco. Da mesma forma, a classe SetorResponsavel "tem uma" lista de Animal e uma lista de PessoaTutora. Essa abordagem favorece a flexibilidade e um baixo acoplamento entre as classes.

2.2. Padrão de Projeto MVC (Model-View-Controller)

A arquitetura do sistema seguiu o padrão MVC para garantir a separação de responsabilidades, conforme solicitado no requisito do projeto.

- **Model:** Representa os dados e as regras de negócio. No projeto, corresponde ao pacote model, contendo as classes Animal, PessoaTutora, SetorResponsavel e Endereco.
- **View:** É a camada de apresentação, responsável pela interface com o usuário. No projeto, corresponde à classe Main (no pacote View), que exibe menus, solicita dados e apresenta resultados via console.
- **Controller:** Atua como o intermediário, recebendo as solicitações da View e manipulando o Model para atendê-las. Corresponde ao pacote controller, que contém a lógica da aplicação, organizada em um UefsAnimaisController e controllers específicos para cada entidade.

2.3. Validação com REGEX

Para garantir a integridade dos dados de entrada, como e-mails e telefones, foram utilizadas Expressões Regulares (Regex). Uma Regex é uma sequência de caracteres que define um padrão de busca. No projeto, ela foi aplicada na classe Validador para criar regras precisas que um campo de texto deve seguir. Por exemplo, a Regex para e-mail verifica a presença de um caractere @ e um domínio válido, enquanto a de telefone verifica a quantidade correta de dígitos, independentemente de formatações como parênteses ou hifens. O uso de Regex permitiu a criação de validações robustas e flexíveis de forma declarativa.

2.4. Persistência de Dados com JSON

Um requisito fundamental do projeto era o armazenamento permanente dos dados. Para isso, foi escolhida a abordagem de serialização de objetos em formato JSON (JavaScript Object Notation).

Para realizar a conversão entre os objetos Java e o formato JSON (processos de serialização e desserialização), foi utilizada a biblioteca Jackson. Uma classe PersistenciaController foi criada para centralizar essa lógica, salvando todo o estado da aplicação em um único arquivo dados_app_animais.json ao fechar e recarregando-o ao iniciar.

3. Metodologia

O desenvolvimento do projeto seguiu um processo iterativo, partindo da modelagem dos dados até a implementação das funcionalidades e testes.

3.1. Requisitos e Funcionalidades

Com base no documento do problema, foram implementados os seguintes requisitos funcionais:

- CRUD completo para as entidades Animal, PessoaTutora e SetorResponsavel
- Validação de dados de entrada, como formatos de e-mail e telefone, e datas não futuras.
- Sistema de Relatórios, incluindo a listagem completa das entidades e as relações de animais por setor e por tutor.
- Lógica de associação para vincular e desvincular animais e tutores a setores.
- Persistência de dados em arquivo.
- Testes de unidade para todas as classes de lógica.

- Documentação Javadoc em todas as classes e métodos.

3.2. Processo de Codificação e Escolhas Técnicas

O desenvolvimento seguiu um processo iterativo, com foco na validação contínua de cada camada. Primeiramente, foi construída a camada Model e, imediatamente após, foram criados os testes de unidade para validar o comportamento de suas classes. Em seguida, foi desenvolvida a camada Controller, onde também foram feitos testes de unidade para garantir o funcionamento da lógica de negócio. A View foi implementada por último, interagindo com os controllers já testados. Finalmente, a camada de Persistência foi adicionada e validada.

Uma das principais escolhas técnicas do projeto foi a estrutura de dados para o armazenamento em memória. Optou-se pelo uso de `HashMap<Integer, T>` nos controllers, uma decisão que garante alta performance nas operações de busca por ID, com uma complexidade de tempo média de $O(1)$, sendo muito mais eficiente que uma busca linear em uma lista, de complexidade $O(n)$.

Ainda no quesito de robustez, a interface com o usuário foi projetada para ser tolerante a falhas. Para isso, foram criados métodos auxiliares na View, como o `lerInteiro()`, que encapsula a leitura de números em um bloco try-catch para tratar a `NumberFormatException`, impedindo que a aplicação encerre caso o usuário digite um texto. De forma semelhante, o método `lerStringObrigatorio()` foi implementado para assegurar que as entradas não fiquem em branco.

3.3. Metodologia de Interação com Inteligência Artificial

Conforme orientado no documento do problema, a Inteligência Artificial Generativa foi utilizada como uma ferramenta de apoio, seguindo as boas práticas recomendadas. A interação ocorreu principalmente nas seguintes áreas:

- **Depuração (Debugging):** A IA foi consultada para obter dicas e explicações sobre mensagens de erro complexas, como as exceções lançadas pela biblioteca Jackson durante a implementação da persistência.
- **Configuração de Dependências:** A IA auxiliou na busca e formatação das dependências do JUnit 5 e do Jackson para o arquivo `pom.xml` do Maven.
- **Exploração de Alternativas:** Foram discutidas com a IA diferentes abordagens de implementação, como as estratégias de persistência (serialização nativa vs. JSON) e as melhores formas de estruturar os testes de unidade.
- **Auxílio na Documentação:** A IA foi utilizada para auxiliar na documentação Javadoc e para revisar a clareza e a consistência dos comentários, sendo todo o conteúdo final revisado e adaptado pelo autor.

3.4. Ficha Técnica

- **Sistema Operacional:** Windows 11
- **Linguagem:** Java
- **IDE:** IntelliJ
- **Build Tool:** Apache Maven
- **Framework de Testes:** JUnit 5
- **Biblioteca de Persistência:** Jackson

- **Formato de Dados: JSON**

4. Resultados e Discussões

4.1. Manual de Uso

O uso da aplicação é guiado por menus de console e segue um fluxo de operações intuitivo. Abaixo estão detalhados os principais passos e regras de negócio do sistema:

1. Iniciando e Encerrando a Aplicação

Ao ser executada, a aplicação automaticamente carrega os dados salvos de sessões anteriores do arquivo `dados_app_animais.json`. Da mesma forma, ao encerrar o programa através da opção "0" no menu principal, todos os dados atuais (novos cadastros, alterações, exclusões) são salvos permanentemente no mesmo arquivo para a próxima utilização.

2. Ordem Recomendada para Cadastro

Para um uso completo das funcionalidades de associação, é recomendado seguir uma ordem lógica de cadastro:

- Primeiro, cadastre os Setores Responsáveis e as Pessoas Tutoras.
- Em seguida, cadastre os Animais.
- Com as entidades já existentes, utilize as funções de associação para criar os vínculos entre elas.

3. Operações de Gerenciamento (CRUD)

Cada menu de gerenciamento (Animal, Tutor e Setor) oferece as operações padrão de: Cadastrar um novo registro, Buscar por ID ou nome, Atualizar um registro existente, Listar todos os registros, gerar Relatórios e Deletar um registro.

4. Regras de Associação e Desassociação

O sistema possui regras para garantir a correta atribuição de responsabilidades:

- Para associar um animal a um setor: É obrigatório que o setor de destino já possua ao menos uma pessoa tutora associada.
- Para desassociar um tutor de um setor: O sistema impede que a última pessoa tutora seja desassociada de um setor caso este ainda possua animais. Essa regra garante que nenhum animal fique sem um responsável no setor.

5. Regras de Remoção

Para garantir a integridade dos dados e evitar a perda de informações por acidente, o sistema implementa regras de proteção que bloqueiam a exclusão de registros que possuem associações ativas:

- Deletar um Setor: Um setor não pode ser deletado se houver qualquer animal ou tutor associado a ele. É necessário primeiro desassociar todos os membros.
- Deletar um Tutor: Uma pessoa tutora não pode ser deletada se estiver associada a qualquer setor. É necessário primeiro desassociá-la.
- Deletar um Animal: Um animal não pode ser deletado se estiver associado a qualquer setor. É necessário primeiro desassociá-lo.

- **Confirmação:** Adicionalmente, toda operação de deleção exige uma confirmação explícita do usuário (1(sim)/2(não)) para evitar a remoção de dados acidentalmente.

4.2. Dados de Entrada e Saída

A aplicação interage com o usuário exclusivamente através do console. Os dados de entrada são fornecidos via texto, incluindo nomes, IDs numéricos para seleção, e opções de menu. O sistema implementa validações para garantir a integridade desses dados, conforme detalhado na metodologia. Os dados de saída são apresentados de duas formas:

- **No Console:** Mensagens de texto formatadas, incluindo confirmações de sucesso ("Animal cadastrado com sucesso!"), mensagens de erro amigáveis ("Falha ao remover. O animal ainda está associado a um setor...") e os relatórios de listagem de dados.
- **Em Arquivo:** O principal dado de saída da aplicação é o arquivo `dados_app_animais.json`, que armazena todo o estado do sistema de forma permanente, garantindo a persistência das informações entre as execuções.

4.3. Testes e Erros Encontrados

A etapa de testes de unidade foi crucial para garantir a qualidade da lógica de negócio do sistema. Utilizando o framework JUnit 5, a estratégia adotada foi criar uma classe de teste para cada classe dos pacotes `model` e `controller`. Cada método de teste foi projetado para validar um comportamento específico, cobrindo tanto os cenários de sucesso quanto as falhas esperadas. A anotação `@BeforeEach` foi usada para que cada teste fosse executado a partir de um estado limpo, com uma nova instância do `controller`.

Durante o desenvolvimento do projeto, foram encontrados e solucionados alguns erros, sendo eles:

- **FAIL_ON_EMPTY_BEANS:** Ocorreu porque as classes de `controller` não mostravam publicamente seus atributos. Foi resolvido configurando o `ObjectMapper` para acessar os campos privados diretamente.
- **Unrecognized field "idade":** Ocorreu porque a idade do animal, sendo um valor calculado, era salva no JSON mas não podia ser carregada de volta. Foi resolvido com a anotação `@JsonIgnore` no método `getIdade()`.
- **Validação de E-mail:** A expressão regular (Regex) inicial para validar e-mails era muito permissiva e aceitava formatos inválidos, como endereços com pontos consecutivos (ex: `nome.sobrenome@email.com`). Após a identificação do problema, a expressão foi substituída por uma versão mais completa.

5. Conclusão

O sistema desenvolvido atendeu aos requisitos propostos, entregando uma solução funcional, robusta e testada para o gerenciamento dos animais da UEFS. A arquitetura MVC garantiu um código limpo e de fácil manutenção, e a implementação da persistência com JSON assegurou que os dados dos usuários sejam mantidos entre as execuções. O uso de testes de unidade dá um alto grau de confiabilidade à lógica de negócio.

O projeto não apenas soluciona um problema prático da universidade, mas também serve como uma aplicação consolidada dos principais conceitos de Programação Orientada a Objetos, padrões de projeto, testes automatizados e

manipulação de dados, contribuindo para o ganho de conhecimento.

5.1. Trabalhos Futuros

Como evolução do projeto, substituir a interface de console por uma interface gráfica utilizando JavaFX ou Swing seria uma boa opção para uma experiência de usuário mais rica.

6. Referências

DEITEL, P.; DEITEL, H. Java: Como Programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

JUNIT TEAM. JUnit 5 User Guide. Disponível em:
<https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 04 set. 2025.

FASTERXML. Jackson Project Home. Disponível em:
<https://github.com/FasterXML/jackson>. Acesso em: 12 out. 2025.